ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

Department of Software Engineering

Section-2 / Goup-4
Course-name: Mobile App Design and Development (SEng3206)

Title: System Design Documentation For Ecommerce Mobile App

Prepared by:

	Name	ID.NO
1.	Eyuel Getachew	ugr/34383/16
2.	Ermiyas Mulugeta	ugr/34317/16
3.	Frehiwet Zerihun	ugr/34470/16
4.	Lidet Admassu	ugr/34816/16
5.	Abel Tagesse	ugr/33810/16

Submitted to : Olyad Temesgen
Date of Submission: May 16, 2026

System Design Documentation

1. System Architecture

High-Level Architecture Overview

The e-commerce application follows a modern three-tier architecture with clear separation of concerns, enabling independent development, testing, and deployment of each component.

Architecture Components

Presentation Layer (Mobile Application)

The presentation layer is responsible for rendering the user interface and handling user interactions. It is built as a cross-platform mobile application using React Native and Expo, targeting both iOS and Android devices. This approach allows for a single codebase, reducing development time and maintenance overhead while providing a native-like user experience.

Framework Stack: - React Native 0.81.5 with Expo SDK 54+: Provides a robust framework for building native mobile applications using JavaScript and React. Expo simplifies the development workflow with pre-built modules and services. - Expo Router for file-based navigation: Enables intuitive and declarative navigation within the application, automatically generating routes based on file structure. - Zustand for client-side state management: A lightweight and fast state management solution that helps manage the application's UI state efficiently and predictably. - Supabase JS SDK for backend integration: Facilitates seamless interaction with the Supabase backend services, including authentication, database access, and real-time subscriptions. **Key Features:** - Cross-platform mobile application (iOS/Android): Ensures broad reach and accessibility across major mobile operating systems. - Responsive UI with native performance:

Delivers a smooth and engaging user experience that adapts to various screen sizes and orientations. - Offline-first cart functionality: Allows users to add items to their cart even without an internet connection, enhancing usability and reliability. - Real-time data synchronization: Provides instant updates to product information, order statuses, and other dynamic content, ensuring users always see the most current data.

Rationale for Technology Choices:

- React Native: Chosen for its ability to build native mobile applications using JavaScript, allowing for code reuse between platforms and faster development cycles compared to separate native development.
- Expo: Selected to streamline the React Native development process by providing a managed workflow, access to native device features without complex native module setup, and simplified deployment.
- Zustand: Opted for its simplicity, small bundle size, and performance benefits for state management, avoiding the boilerplate often associated with larger state management libraries.
- Supabase JS SDK: Integrates directly with the Supabase backend, offering a convenient and efficient way to interact with the database, authentication, and storage services without building custom API clients.

Application Layer (Backend API)

The application layer serves as the central hub for business logic, handling requests from the mobile application, processing them, and interacting with the data layer. It is implemented as a RESTful API using Node.js and Express.js, providing a scalable and efficient backend for the e-commerce platform.

Technology Stack: - Node.js runtime environment: A powerful and efficient JavaScript runtime that allows for building scalable network applications. - Express.js web framework: A minimalist and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. - RESTful API design principles: Adherence to Representational State Transfer (REST) principles ensures a stateless, client-server architecture with a uniform interface, making the API easy to understand and consume. - JWT authentication middleware: JSON Web Tokens (JWT) are used for secure,

stateless authentication, allowing the API to verify the authenticity of requests without needing to query a database on every call. **Responsibilities:** - Request validation and sanitization: Ensures that all incoming requests are well-formed and free from malicious content, protecting the backend from common vulnerabilities. - Business logic processing: Contains the core logic for handling e-commerce operations, such as product management, order processing, and user profile updates. - Database query optimization: Manages efficient interaction with the database, including query construction, indexing strategies, and connection pooling to ensure optimal performance. - Error handling and logging: Implements a centralized error handling mechanism and comprehensive logging to monitor application health, debug issues, and ensure system stability.

Rationale for Technology Choices:

- Node.js: Selected for its non-blocking, event-driven architecture, which is ideal for building high-performance, scalable APIs capable of handling a large number of concurrent connections.
- Express.js: Chosen for its simplicity, flexibility, and extensive middleware ecosystem, which accelerates API development and provides a solid foundation for building RESTful services.
- RESTful API Design: Adopted for its industry-standard approach to web service design, promoting interoperability, scalability, and ease of integration with various client applications.
- JWT Authentication: Implemented for its stateless nature, which reduces server load and simplifies authentication across distributed systems, while providing a secure method for token-based authorization.

Data Layer (Supabase Platform)

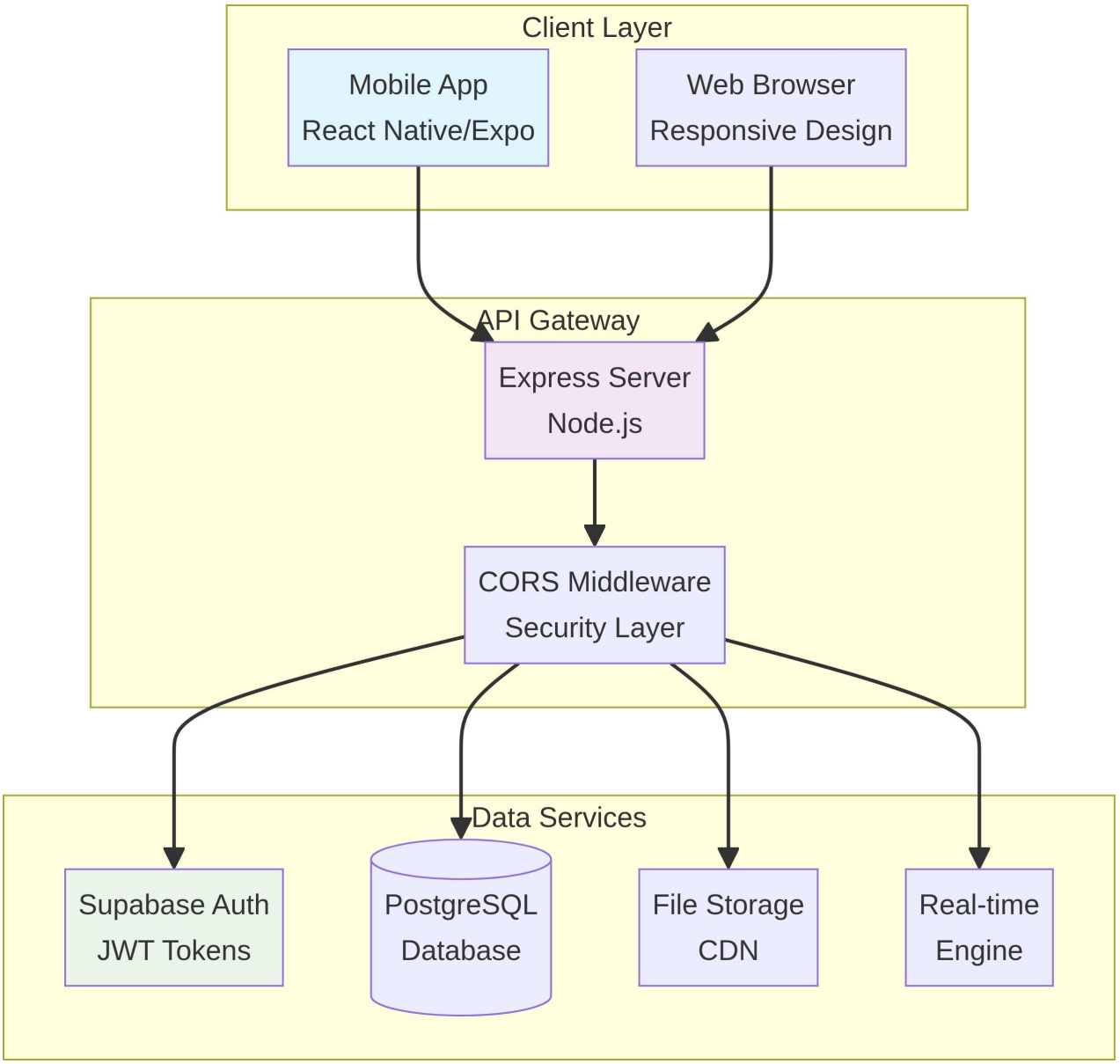
The data layer leverages Supabase, an open-source Firebase alternative, to provide a robust and scalable backend infrastructure. Supabase offers a suite of services including a PostgreSQL database, authentication, real-time capabilities, and file storage, all managed through a unified platform. This simplifies backend development and allows the application to focus on core features.

Database Services: - PostgreSQL relational database: A powerful, open-source object-relational database system known for its reliability, feature robustness, and performance. It forms the core of the data storage for the e-commerce platform. - Row Level Security (RLS) policies: Implemented to enforce fine-grained access control on database rows, ensuring that users can only access data they are authorized to see or modify. This is crucial for multi-tenant applications and user privacy. - Real-time subscriptions: Supabase provides real-time capabilities, allowing the mobile application to subscribe to database changes and receive instant updates. This is used for features like live order tracking or real-time inventory updates. - File storage for product images: Supabase Storage handles the secure and scalable storage of product images and other media assets, with CDN integration for fast content delivery. **Security Features:** - Authentication service integration: Supabase Auth provides a comprehensive authentication solution, supporting various sign-in methods and integrating seamlessly with the PostgreSQL database for user management. - Role-based access control: Leverages PostgreSQL roles and RLS policies to define and enforce different access levels for users (e.g., customer, admin), ensuring that actions are restricted based on their assigned roles. - Data encryption at rest and in transit: All data stored in the PostgreSQL database is encrypted at rest, and all communication between the application and Supabase is secured using SSL/TLS encryption, protecting sensitive information. - Automated backup and recovery: Supabase manages automated daily backups and provides point-in-time recovery capabilities, ensuring data durability and minimizing potential data loss.

Rationale for Technology Choices:

- Supabase: Chosen as a comprehensive backend-as-a-service (BaaS) solution that provides a managed PostgreSQL database, authentication, and storage, significantly accelerating development and reducing operational overhead.
- PostgreSQL: Selected for its advanced features, ACID compliance, and extensibility, making it suitable for complex relational data and supporting features like JSONB for flexible data structures.
- Row Level Security (RLS): Essential for implementing robust security directly at the database level, preventing unauthorized data access even if application-level security is compromised.
- Real-time Capabilities: Crucial for modern e-commerce applications that require instant updates and interactive user experiences, such as live inventory counts or order status changes.

System Architecture Diagram



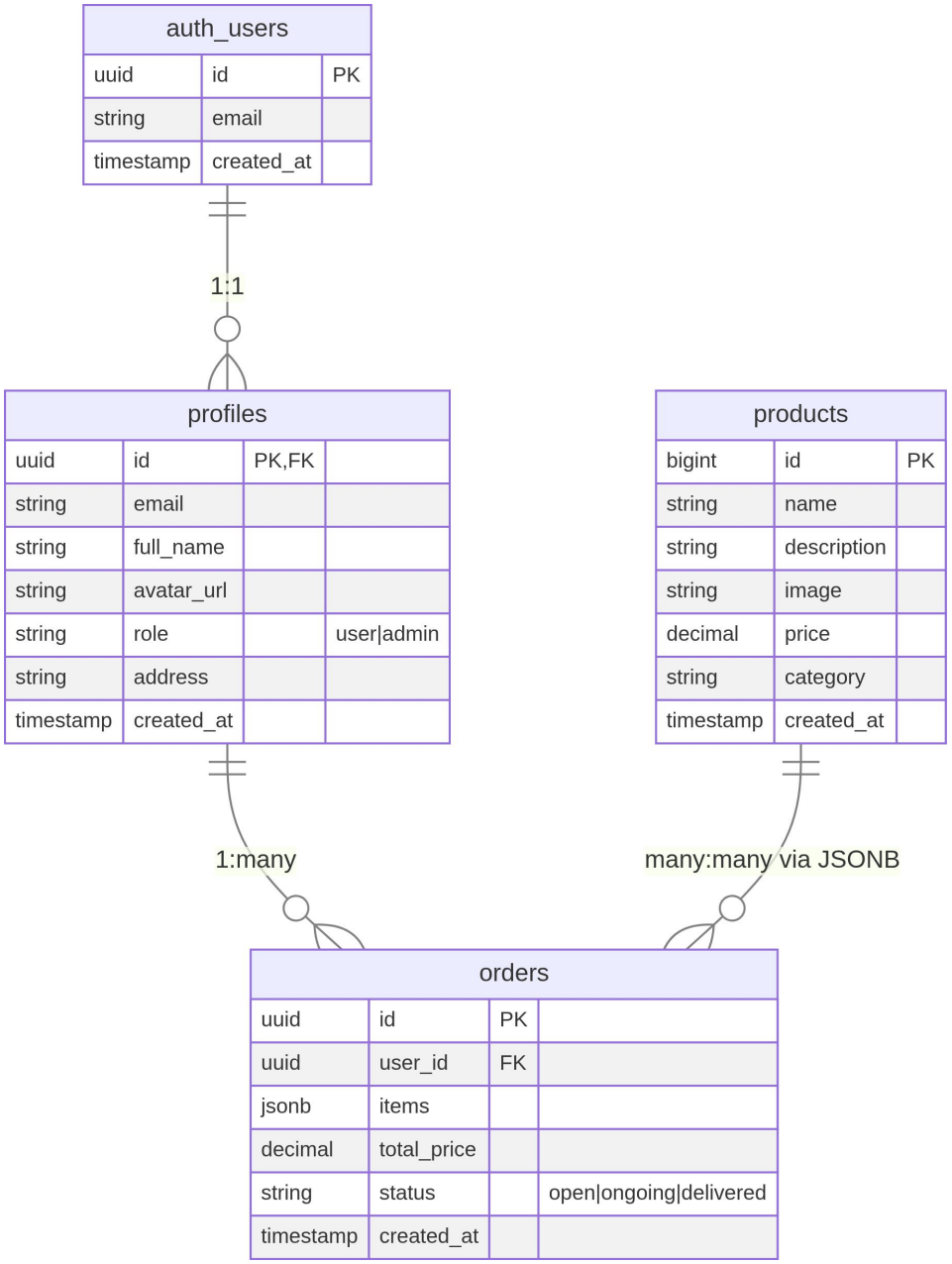
Data Flow Architecture

Component	Input	Processing	Output
Mobile App	User interactions	State management	API requests
API Server	HTTP requests	Business logic	Database queries
Database	SQL queries	RLS policies	Filtered data
Real-time Engine	Data changes	Event broadcasting	Live updates

---

2. Database Design (ERD)

Entity-Relationship Diagram



Entity Descriptions

auth.users Table

The `auth.users` table is a fundamental component of the authentication system, managed directly by Supabase Auth. It stores essential user credentials and metadata, acting as the primary source of truth for user identities within the platform. This table is automatically populated and managed by Supabase's authentication services, abstracting away much of the complexity of user management.

Purpose: This table serves as the core authentication record, managed by the Supabase Auth service. It securely stores user login information and acts as the central identity provider for the application. **Relationships:** It maintains a one-to-one relationship with the ` profiles` table, where each authenticated user has a corresponding profile entry for extended metadata. **Key Constraints:** The ` id` column is the primary key, ensuring unique identification for each user. The ` email` column has a unique constraint to prevent duplicate user accounts and facilitate login. **Schema:** - ` id` (UUID, Primary Key): A universally unique identifier for each user, automatically generated upon registration. This serves as the immutable identifier for the user across the system. - ` email` (TEXT, Unique): The user's email address, used for login and communication. The unique constraint ensures that no two users can register with the same email. - ` created_at` (TIMESTAMP): Automatically records the timestamp when the user account was created, useful for auditing and user analytics.

Technical Considerations:

- Security: Supabase Auth handles password hashing and storage, ensuring industry-standard security practices are applied without direct application intervention.
- Integration: Seamlessly integrates with other Supabase services, such as Row Level Security (RLS) and storage, to provide a cohesive and secure backend.
- Scalability: Designed to handle a large number of users, with Supabase managing the underlying infrastructure for high availability and performance.

profiles Table

The `profiles` table stores additional user-specific information that is not directly handled by the authentication service. It extends the basic user data from `auth.users` with details relevant to the e-commerce application, such as full name, avatar URL, and shipping address. This separation allows for flexible management of user profiles without modifying the core authentication records.

Purpose: This table is designed to hold extended user profile information and manage user roles within the application. It complements the ` auth.users` table by storing application-specific user data. **Relationships:** It has a one-to-one foreign key relationship with ` auth.users` via the ` id` column, linking each profile to an authenticated user. It is also referenced by the ` orders` table, associating orders with specific user profiles. **Security:** Row Level Security (RLS) policies are applied to this table to restrict access, ensuring that users can only view and modify their own profile data, and administrators have appropriate access. **Schema:** - ` id` (UUID, Primary/Foreign Key): Serves as both the primary key for the profiles table and a foreign key referencing the ` id` in the ` auth.users` table, maintaining the one-to-one relationship. - ` email` (TEXT): The user's email address, denormalized from ` auth.users` for improved query performance in scenarios where joining with ` auth.users` is not strictly necessary. - ` full_name` (TEXT): The user's full display name, used for personalization and communication within the application. - ` avatar_url` (TEXT): A URL pointing to the user's profile picture, allowing for visual identification. - ` role` (TEXT): Defines the access level of the user (e.g., 'user' for regular customers, 'admin' for administrative personnel). This field is crucial for role-based access control. - ` address` (TEXT): The user's primary shipping address, used for order fulfillment. - ` created_at` (TIMESTAMP): Records the timestamp when the user profile was created.

Technical Considerations:

- Data Denormalization: The `email` field is denormalized to optimize read performance for common queries that require user email without needing to join the `auth.users` table.
- Role-Based Access Control (RBAC): The `role` field is central to implementing RBAC, allowing the system to grant or deny access to certain features or data based on the user’s assigned role.

- Privacy: RLS policies are critical for protecting user privacy by ensuring that sensitive profile information is only accessible to authorized individuals.

products Table

The `products` table is the central repository for all product-related information within the e-commerce platform. It stores details such as product name, description, pricing, imagery, and categorization. This table is designed to support efficient querying for product listings, search functionalities, and inventory management.

Purpose: This table serves as the comprehensive product catalog, storing all necessary information about items available for sale, including inventory details. **Relationships:** Products are referenced by orders through a JSONB structure within the ` orders` table, allowing for flexible and efficient storage of order line items without complex many-to-many join tables. **Indexing:** The table is strategically indexed to optimize common operations such as product search, filtering by category, and price range queries, ensuring fast retrieval times for users. **Schema:** - ` id` (BIGINT, Primary Key): A unique, auto-generated identifier for each product. This is a ` BIGINT` to accommodate a large number of products.
- ` name` (TEXT): The display name of the product, optimized for search and user readability. - ` description` (TEXT): A detailed textual description of the product, providing comprehensive information to customers. - ` image` (TEXT): A URL pointing to the primary image of the product, typically hosted on a CDN for fast loading. - ` price` (DECIMAL): The selling price of the product, stored as a
` DECIMAL` with two decimal places to ensure precision in financial calculations. - ` category` (TEXT): A categorical label for the product (e.g., 'Electronics', 'Apparel', 'Home Goods'), used for filtering and navigation. - ` created_at` (TIMESTAMP): Records the timestamp when the product was added to the catalog.

Technical Considerations:

- JSONB for Order Relationships: Using JSONB in the `orders` table to store product details within an order simplifies the data model for many-to-many relationships, especially when product details at the time of order need to be preserved (e.g., price changes).
- Search Optimization: Indexes on `name` and `category` fields are crucial for efficient product search and filtering, enhancing the user experience.

- Data Integrity: Constraints ensure that product prices are valid and that essential fields like `name` are not null, maintaining data quality.

orders Table

The `orders` table is critical for tracking all customer purchases, managing their lifecycle from placement to delivery. It stores comprehensive details about each order, including the associated user, the items purchased, the total price, and the current status of the order. The use of a JSONB field for `items` provides flexibility in storing diverse product details at the time of purchase.

Purpose: This table is dedicated to storing customer order records and facilitating status tracking throughout the order fulfillment process. **Relationships:** It establishes a foreign key relationship with the ` profiles` table via ` user_id` , linking each order to a specific customer profile. This ensures that all orders are associated with a registered user. **Data Structure:** The ` items` field utilizes a JSONB data type, allowing for a flexible and schema-less storage of an array of ordered products, including their quantities and prices at the time of order. This is particularly useful for handling product variations and historical pricing. **Schema:**
- ` id` (UUID, Primary Key): A unique, universally unique identifier for each order, ensuring no two orders have the same ID. - ` user_id` (UUID, Foreign Key): A foreign key referencing the ` id` in the ` profiles` table, indicating which user placed the order. - ` items` (JSONB): An array of JSON objects, each representing a product within the order. Each object typically includes ` product_id` , ` quantity` , and ` price` at the time of purchase. - ` total_price` (DECIMAL): The calculated total cost of the order, including all items, taxes, and shipping (if applicable).
Stored as ` DECIMAL` for financial accuracy. - ` status` (TEXT): Represents the current lifecycle state of the order (e.g., 'open', 'ongoing', 'delivered'). This field is crucial for order management and customer communication. - ` created_at` (TIMESTAMP): Records the timestamp when the order was placed, providing a historical record.

Technical Considerations:

- JSONB Flexibility: The `items` JSONB field allows for dynamic schema evolution for product details within an order, which is advantageous when product attributes might change over time but historical order data needs to remain consistent.

- Order Status Management: The `status` field is integral to the order fulfillment workflow, enabling tracking and updates as the order progresses through different stages.
- Referential Integrity: The foreign key constraint on `user_id` ensures that every order is linked to a valid user profile, maintaining data consistency.
- Auditing: The `created_at` timestamp is vital for auditing, reporting, and analyzing order patterns over time.

Database Constraints and Indexes

Constraint Type	Table	Columns	Purpose
Primary Key	All tables	id	Unique record identification
Foreign Key	profiles, orders	user_id	Referential integrity
Unique	auth.users	email	Prevent duplicate accounts
Check	profiles	role	Valid role values only
Check	orders	status	Valid status values only

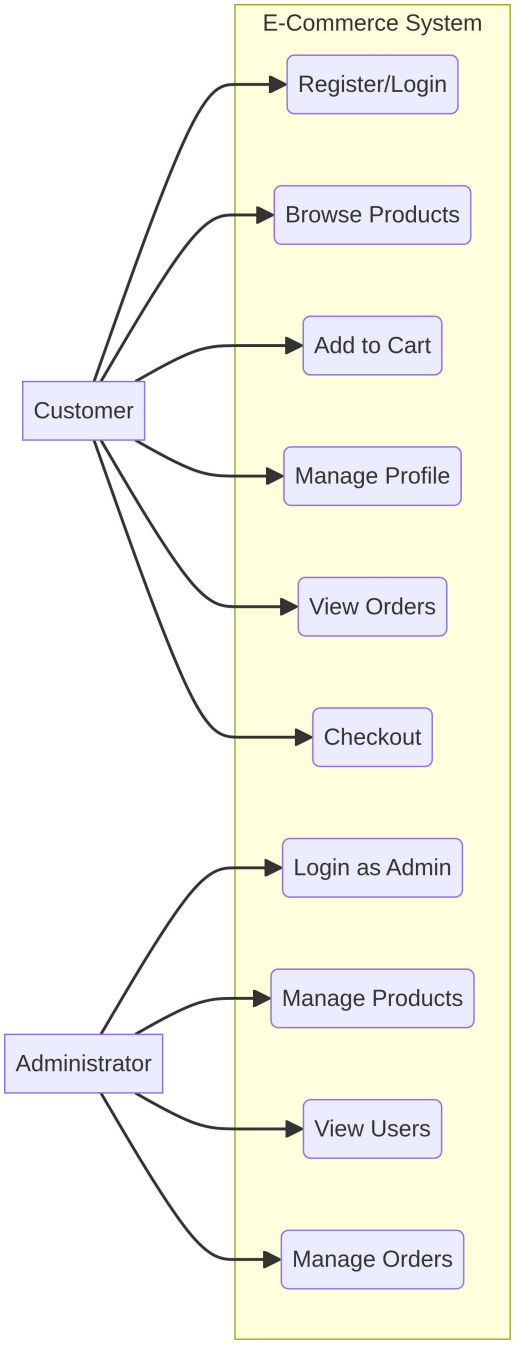
Indexing Strategy

Index Type	Table	Columns	Use Case
B-tree	products	name	Text search optimization
B-tree	products	category	Category filtering
B-tree	products	price	Price range queries
B-tree	orders	user_id	User order history
B-tree	orders	status	Status-based filtering
GIN	orders	items	JSONB array operations

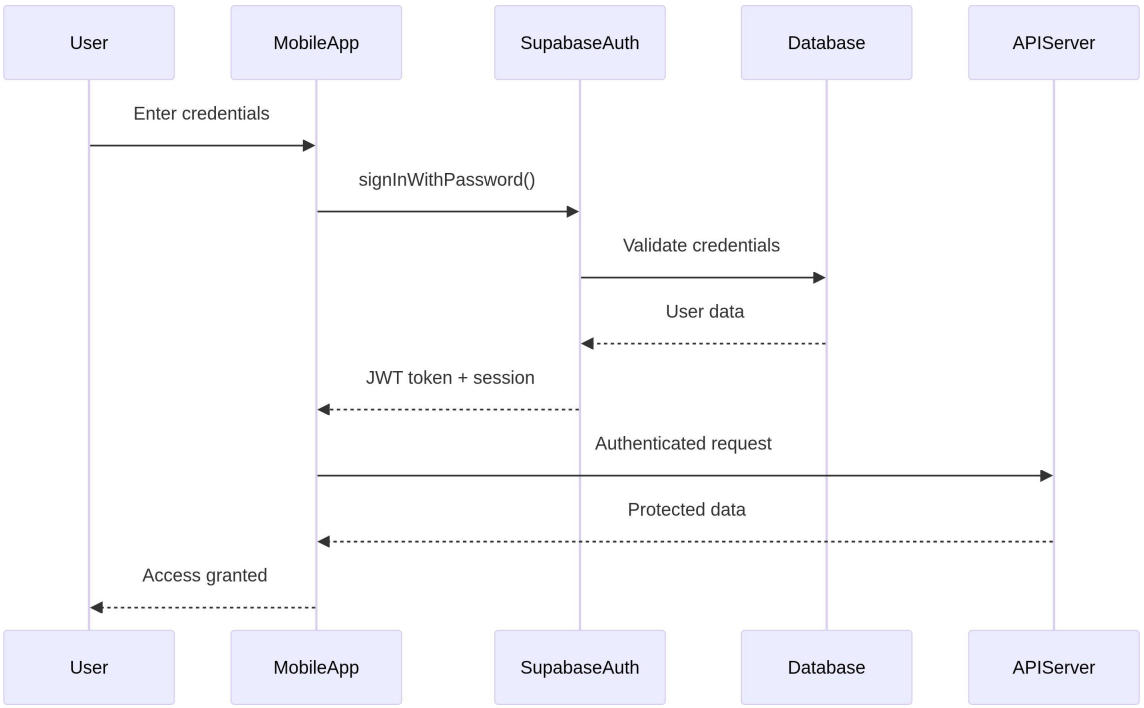
---

3. UML Diagrams

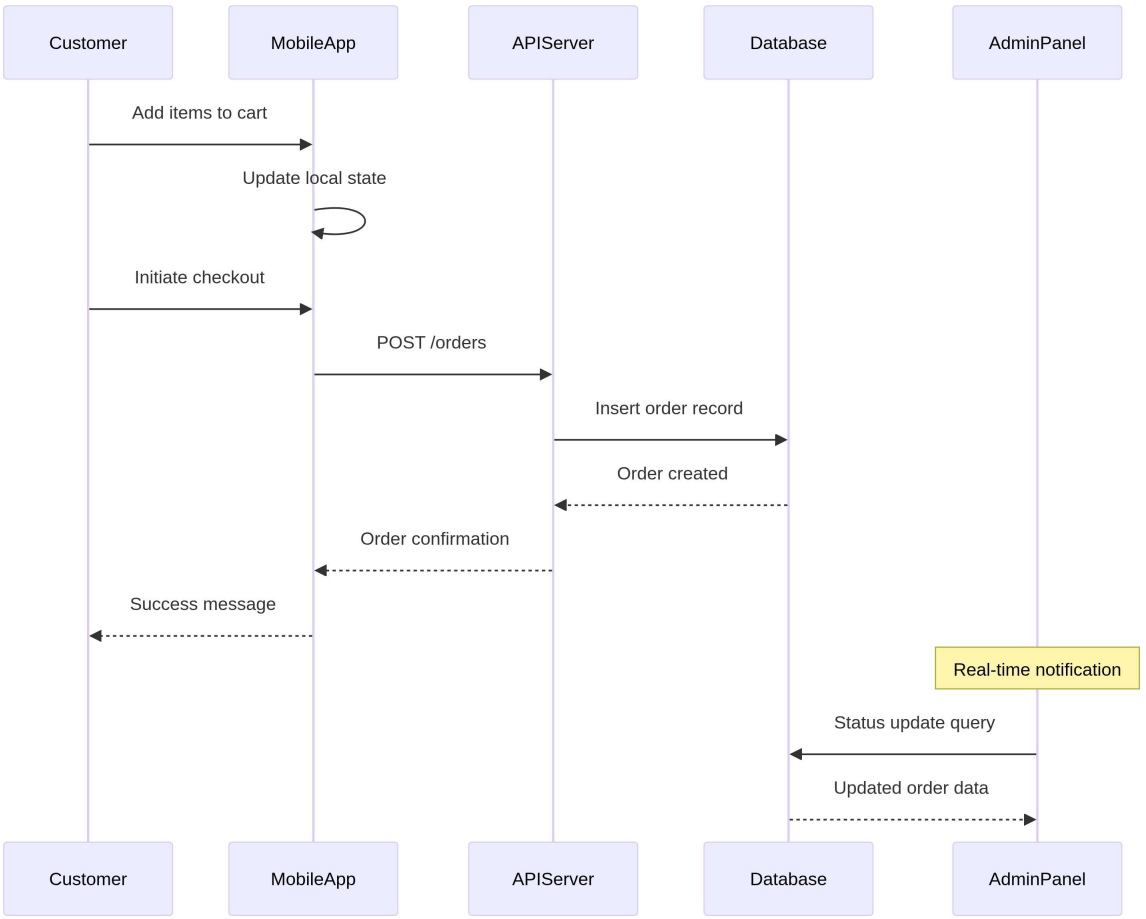
Use Case Diagram



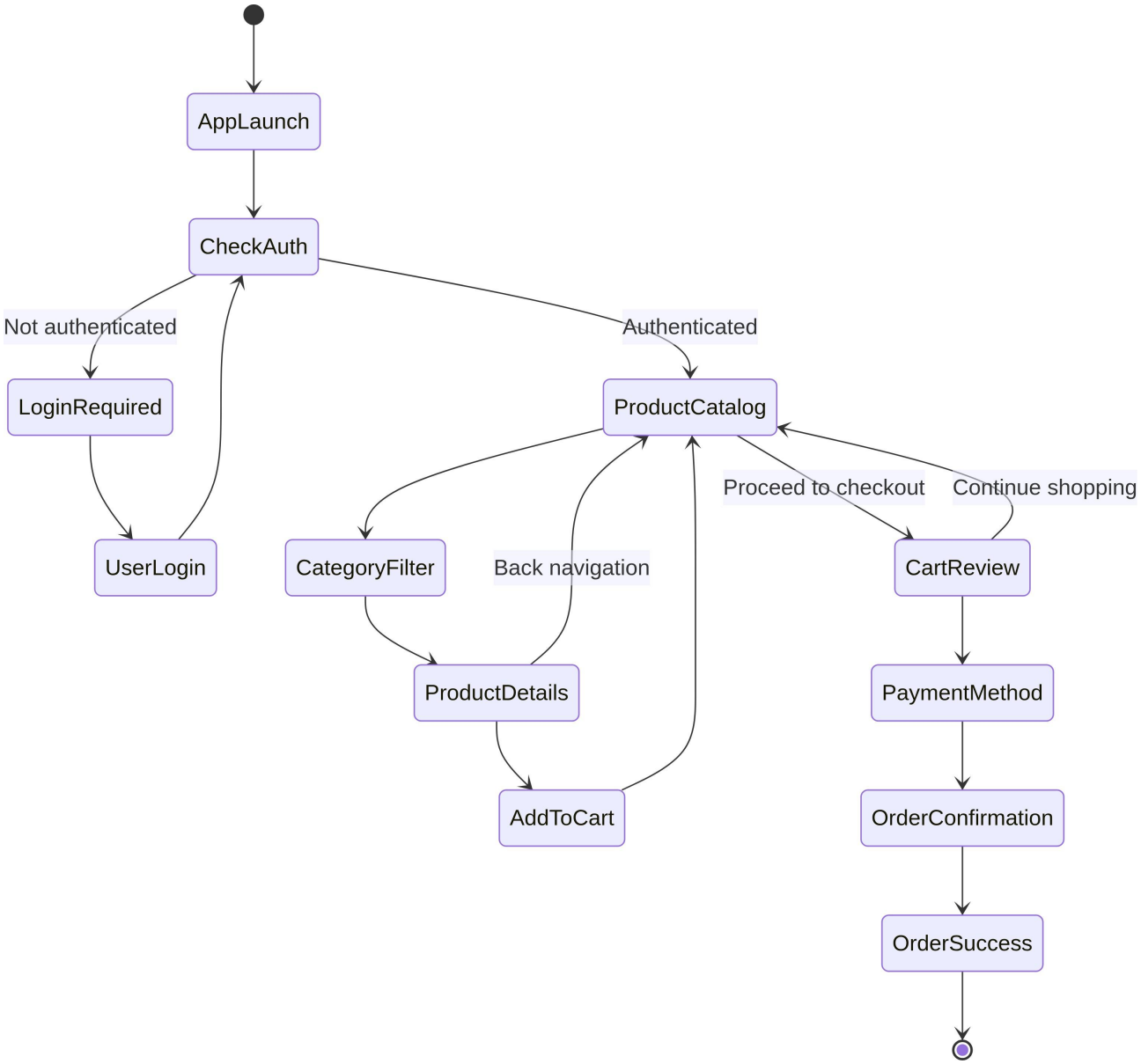
Sequence Diagram - User Authentication Flow



Sequence Diagram - Order Processing Flow



Activity Diagram - Complete Shopping Workflow



4. UI/UX Design Specifications

Screen Hierarchy and Navigation

```
Application Structure
├── Authentication Flow
│   ├── Splash Screen
│   ├── Welcome Screen
│   ├── Login Screen
│   └── Registration Screen
├── Main Application (Tab-based)
│   ├── Home (Product Catalog)
│   ├── Cart (Shopping Cart)
│   ├── Profile (User Account)
│   └── Admin (Conditional)
└── Modal Overlays
    ├── Product Details
    ├── Checkout Wizard
    └── Order Confirmation
```

Screen Specifications

Product Catalog (Home Screen)

The Product Catalog, serving as the application's home screen, is designed to provide users with an intuitive and visually appealing way to browse available products. It features a dynamic grid layout that adapts to various screen sizes, ensuring a consistent experience across different mobile devices. Horizontal scrolling categories allow for easy navigation between product types, while a prominent search bar facilitates quick product discovery.

Layout: The screen utilizes a responsive grid-based layout for displaying products, complemented by horizontal scrolling categories at the top. This design maximizes product visibility while maintaining a clean and organized interface. **Components:** - Category filter tabs: Interactive tabs that allow users to filter products by category, enhancing discoverability and navigation. - Search bar with autocomplete: A prominent search input field that provides real-time suggestions as the user types, improving search efficiency. - Product cards (image, title, price):

Each product is represented by a card displaying its image, name, and price, designed for quick scanning and visual appeal. - Pull-to-refresh functionality: Allows users to refresh the product list by pulling down on the screen, ensuring they always see the latest product data. **Interactions:** - Tap product card → Product detail modal: Tapping on any product card opens a modal displaying detailed information about the selected product. - Swipe categories → Filter products: Swiping horizontally across the category tabs filters the displayed products according to the selected category. - Pull down → Refresh product data: Performing a pull-down gesture on the product list triggers a refresh, fetching the latest product information from the server.

Technical Implementation Details:

- Data Fetching: Products are fetched from the backend API using pagination to optimize performance and reduce initial load times. Infinite scrolling can be implemented to load more products as the user scrolls.
- State Management: Zustand is used to manage the state of the product catalog, including the currently selected category, search query, and loaded products.
- Image Loading: Product images are loaded asynchronously and cached to improve performance and reduce network requests.
- Accessibility: The UI components are designed with accessibility in mind, ensuring proper labeling and navigation for users with disabilities.

Product Detail Modal

The Product Detail Modal provides an immersive view of a selected product, allowing users to examine its features, view multiple images, and make purchase decisions. It is designed as a full-screen overlay to maintain focus on the product and offers intuitive interactions for exploring product information.

Layout: This is a full-screen modal that presents product information in a detailed and organized manner, featuring an image carousel at the top and product specifications below. **Components:** - Image gallery with zoom: Allows users to view multiple high-resolution images of the product and zoom in for closer inspection. - Product title and description: Clearly displays the product name and a comprehensive description, highlighting key features and benefits. - Price and availability: Shows the current price of the product and its stock status (e.g., 'In

Stock', 'Low Stock', 'Out of Stock'). - Quantity selector: An interactive component that enables users to specify the desired quantity of the product before adding it to the cart. - Add to cart button: A prominent call-to-action button that adds the selected product and quantity to the user's shopping cart. **Interactions:** - Swipe images → Gallery navigation: Users can swipe horizontally across the image gallery to browse through different product images. - Tap quantity → Increment/decrement: Tapping on the quantity selector allows users to increase or decrease the number of items they wish to purchase. - Add to cart → Toast notification + cart update: Upon adding an item to the cart, a temporary toast notification confirms the action, and the shopping cart icon updates to reflect the new item count.

Technical Implementation Details:

- Data Binding: Product details are dynamically loaded from the API based on the selected product ID, ensuring up-to-date information.
- State Management: Zustand manages the state of the quantity selector and the 'Add to Cart' button, reflecting real-time changes.
- Image Optimization: Images in the gallery are optimized for mobile viewing, with lazy loading and responsive sizing to ensure fast load times and smooth transitions.
- User Feedback: Toast notifications provide non-intrusive feedback to the user, confirming actions like adding items to the cart.

Shopping Cart Screen

The Shopping Cart Screen provides users with a comprehensive overview of the items they have selected for purchase. It allows for review, modification of quantities, removal of items, and a clear display of the total cost before proceeding to checkout. The design prioritizes clarity and ease of use, ensuring a smooth transition from product selection to purchase.

Layout: The screen presents a list view of selected items, each with its own card, and a persistent summary footer displaying the total cost. This layout ensures that users can easily manage their cart while always being aware of the running total. **Components:** - Cart item list with images: Each item in the cart is displayed with its image, name, and price, providing visual confirmation of selected products. -

Quantity controls per item: Interactive controls (e.g., plus/minus buttons) allow users to adjust the quantity of each item directly within the cart. - Item total calculations: Displays the subtotal for each item based on its price and quantity. - Cart summary (subtotal, tax, total): A clear breakdown of the total cost, including subtotal, estimated taxes, and the final total amount, presented in the footer. - Checkout button: A prominent call-to-action button that initiates the checkout process. **Interactions:** - Swipe item → Remove option: Users can swipe left or right on a cart item to reveal a 'Remove' option, allowing for easy deletion of unwanted items. - Tap quantity → Edit modal: Tapping on the quantity control might open a small modal or a direct input field for precise quantity adjustments. - Checkout tap → Validation and navigation: Tapping the 'Checkout' button triggers a validation process for the cart contents and then navigates the user to the first step of the checkout flow.

Technical Implementation Details:

- Local State Management: The shopping cart state is primarily managed client-side using Zustand to provide an immediate and responsive user experience. Changes are synchronized with the backend during checkout.
- Price Calculation Logic: All price calculations (item subtotals, total cart value) are performed client-side for responsiveness and re-validated on the server during the checkout process to prevent discrepancies.
- Persistence: Cart contents are persisted locally (e.g., using `AsyncStorage` in React Native) to ensure that items remain in the cart even if the user closes the app.
- Backend Synchronization: When the user proceeds to checkout, the entire cart content is sent to the backend API for final validation, inventory checks, and order creation.

Checkout Flow (Multi-step Modal)

The checkout process is implemented as a multi-step modal to guide users through the necessary stages of completing a purchase. This structured approach minimizes cognitive load and reduces the likelihood of errors, ensuring a smooth and secure transaction. Each step focuses on a specific aspect of the checkout, from reviewing the cart to confirming the order.

Step 1 - Cart Review: - Item list with final quantities: Presents a final summary of all items in the cart, including their quantities and individual prices, allowing users to verify their selection. - Remove/edit options: Provides options to remove items or adjust quantities directly within this step, offering flexibility before proceeding.
- Subtotal display: Clearly shows the subtotal of all items, excluding taxes and shipping, to give users an initial cost overview. **Step 2 - Shipping Information:** - Address form fields: Collects necessary shipping details, including street, city, state, zip code, and country, with clear input fields and labels. - Save for future use option: Allows users to save their shipping address for quicker checkout in subsequent purchases. - Validation feedback: Provides real-time feedback on address validity, guiding users to correct any errors. **Step 3 - Payment Method:** - Payment option selection: Offers various payment methods (e.g., credit card, digital wallets) for users to choose from. - Security indicators: Displays visual cues (e.g., padlock icons, PCI compliance logos) to assure users of the security of their payment information. - Terms acceptance: Requires users to accept the terms and conditions before finalizing the purchase. **Step 4 - Order Confirmation:** - Final order summary: A comprehensive summary of the entire order, including shipping details, selected payment method, and the final total price. - Place order button: The final call-to-action to submit the order to the backend. - Loading states: Provides visual feedback (e.g., spinners) while the order is being processed, preventing users from making multiple submissions.

Technical Implementation Details:

- Step-by-Step Navigation: The modal uses a controlled navigation flow, ensuring users complete each step sequentially. Progress indicators can be used to show the current step.
- Form Validation: Client-side validation is performed at each step to provide immediate feedback, with server-side re-validation during order submission for data integrity.
- Payment Gateway Integration: Integration with a secure payment gateway (e.g., Stripe, PayPal) is handled on the back end to process payments securely and comply with PCI DSS standards.
- Error Handling: Robust error handling is implemented to gracefully manage issues during any step of the checkout process, providing clear messages to the user and logging errors for debugging.

Admin Dashboard

The Admin Dashboard provides a dedicated interface for administrators to manage various aspects of the e-commerce platform. This includes product management, order tracking, user oversight, and access to analytics. The dashboard is secured with role-based access control, ensuring that only authorized personnel can access and perform administrative operations.

Layout: The Admin Dashboard features a dedicated interface with a clear layout designed for efficient management. It typically includes a sidebar for navigation between different administrative sections and a main content area for displaying data and forms. **Components:** - Product management form: Allows administrators to add new products, edit existing product details (name, description, price, image, category), and manage inventory levels. - Order status dashboard: Provides a real-time overview of all customer orders, enabling administrators to track order statuses, update fulfillment details, and resolve issues. - User management (view only): Offers a read-only view of registered users, including their profiles and order history. This is typically restricted to prevent unauthorized modifications to user accounts. - Analytics summary: Displays key performance indicators (KPIs) and summarized data related to sales, popular products, and user activity, helping administrators make informed business decisions. **Security:** Role-based access control (RBAC) is strictly enforced, ensuring that only users with the `admin` role can access the dashboard and its functionalities. This prevents unauthorized access to sensitive administrative tools and data.

Technical Implementation Details:

- Backend API Integration: The dashboard interacts with protected API endpoints that require administrator authentication and authorization for all management operations.
- Data Visualization: Integrates with charting libraries or data visualization tools to present analytics summaries in an easily digestible format.
- Audit Trails: All administrative actions, especially those involving data modification, are logged to provide an audit trail for security and compliance purposes.
- Responsive Design: While primarily for desktop use, the dashboard is designed to be responsive, allowing for basic management tasks on tablets or larger mobile devices.

Design System Specifications

Color Palette

Color Name	Hex Code	Usage
Primary Blue	#007AFF	Buttons, links, active states
Secondary Gray	#6C757D	Secondary text, borders
Success Green	#28A745	Confirmations, positive actions
Warning Yellow	#FFC107	Warnings, admin elements
Danger Red	#DC3545	Errors, destructive actions
Background	#FFFFFF	Primary background
Surface	#F8F9FA	Cards, secondary backgrounds

Typography Scale

Text Style	Font Size	Font Weight	Usage
Headline 1	24px	Bold	Screen titles
Headline 2	20px	Bold	Section headers
Body Large	16px	Regular	Primary content
Body	14px	Regular	Secondary content
Caption	12px	Regular	Metadata, labels
Button	16px	Medium	Button text

Component Specifications

Component	Height	Padding	Border Radius	Shadow
Button Primary	44px	16px horizontal	8px	Subtle drop shadow
Button Secondary	44px	16px horizontal	8px	Border only
Card	Variable	16px	12px	Light shadow
Input Field	48px	12px	8px	Border focus state
Tab Bar	60px	8px vertical	0px	Top border

5. API Design Specifications

RESTful API Architecture

Authentication Endpoints POST

/auth/login

This endpoint facilitates user authentication by verifying provided credentials and, upon successful validation, issues a JSON Web Token (JWT) for subsequent authenticated requests. It is a critical component for securing access to protected resources within the e-commerce platform.

Login User
Purpose: To authenticate user credentials (email and password) and establish a secure session by returning a JWT access token and a refresh token.

Endpoint: /auth/login

Method: POST

Headers: Content-Type: application/json

Request Body

JSON
```
{
  "email": "user@example.com",
  "password": "securepassword123"
}
```

Fields

email *(string, required)*: The user's registered email address

password *(string, required)*: The user's password.

Response (200 OK)

JSON
```
{
  "user": {
    "id": "uuid-string",
    "email": "user@example.com",
    "role": "user"
  },
  "session": {
    "access_token": "jwt-token",
    "refresh_token": "refresh-jwt",
    "expires_at": 1640995200
  }
}
```
Fields

user *(object)*: Contains basic user information.

id *(string)*: Unique identifier of the authenticated user.

email *(string)*: Email address of the authenticated user.

role *(string)*: Role of the authenticated user (e.g., 'user', 'admin').

session *(object)*: Contains session-related tokens.

access_token *(string)*: JWT used for authenticating subsequent API requests.

refresh_token *(string)*: Token used to obtain a new access token when the current one expires.

expires_at *(integer)*: Unix timestamp indicating when the access token expires.

Error Responses

400 Bad Request

Returned if the provided credentials are invalid (e.g., incorrect email format, wrong password).

JSON

```
{
  "error": {
    "code": "INVALID_CREDENTIALS",
    "message": "Invalid email or password"
  }
}
```

429 Too Many Requests

Returned if the user exceeds the rate limit for login attempts, preventing brute-force attacks.

JSON

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many login attempts, please try again later"
  }
}
```

Security Considerations:

- Password Hashing: Passwords are never stored in plain text. They are hashed using a strong, one-way cryptographic function (e.g., bcrypt) before being stored in the database.
- JWT Security: Access tokens are short-lived to minimize the impact of token compromise. Refresh tokens are used to obtain new access tokens without requiring re-authentication, and they are typically longer-lived and stored more securely.
- Rate Limiting: Implemented to prevent brute-force attacks on user accounts by limiting the number of login attempts within a specific time window.
- HTTPS Enforcement: All API communication, especially authentication, is enforced over HTTPS to ensure data encryption in transit and protect against man-in-the-middle attacks.

POST /auth/register

This endpoint allows new users to create an account within the e-commerce platform. Upon successful registration, a new user entry is created in the `auth.users` and `profiles` tables, and a confirmation message is returned. This process is designed to be straightforward while ensuring necessary data collection and security measures.

Register User
Purpose: To create a new user account by registering a unique email, password, and optional profile details.

- **Endpoint:** /auth/register
- **Method:** POST
- **Headers:** Content-Type: application/json

Request Body

JSON

```
{
  "email": "newuser@example.com",
  "password": "securepassword123",
  "full_name": "John Doe",
  "accept_terms": true
}
```

Fields

- email *(string, required)*: The desired email address for the new account. Must be unique.
- password *(string, required)*: The password for the new account. Should meet predefined complexity requirements.
- full_name *(string, optional)*: The user's full name, which will be stored in the profiles table.
- accept_terms *(boolean, required)*: A flag indicating the user's acceptance of the terms of service and privacy policy.

Response (201 Created)

JSON

```
{
  "user": {
    "id": "uuid-string",
    "email": "newuser@example.com"
  },
  "message": "Account created successfully"
}
```

Fields

- user *(object)*: Contains basic information about the newly created user.
    - id *(string)*: Unique identifier of the new user.
    - email *(string)*: Email address of the new user.
- message *(string)*: A confirmation message indicating successful account creation.

Error Responses

400 Bad Request

Returned if the request body is invalid (e.g., missing required fields, invalid email format, password not meeting complexity requirements, terms not accepted, email already exists).

JSON

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Email already registered"
  }
}
```

429 Too Many Requests

Returned if the user exceeds the rate limit for registration attempts.

JSON

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many registration attempts, please try again later"
  }
}
```

Technical Considerations:

- Email Verification: After registration, an email verification flow can be initiated to confirm the user's email address, enhancing security and preventing spam accounts.
- Password Policy: The API enforces a strong password policy (e.g., minimum length, inclusion of special characters, numbers, and mixed case) to protect user accounts.
- Transactional Integrity: The creation of the user in `auth.users` and the corresponding profile in `profiles` should be handled as a single atomic transaction to ensure data consistency.
- CAPTCHA/reCAPTCHA: To prevent automated bot registrations, CAPTCHA or reCAPTCHA integration can be added to the registration flow.

Product Management Endpoints

GET /products

This endpoint allows clients to retrieve a paginated list of products from the catalog. It supports various query parameters for filtering, searching, and sorting, enabling flexible product discovery. This is the primary endpoint for displaying products to users in the mobile application.

Get Products List

Purpose: To retrieve a paginated list of products, with options for filtering, searching, and sorting.

- **Endpoint:** /products
- **Method:** GET

Query Parameters

- page *(integer, default:* 1*)*: The page number of the results to retrieve. Used for pagination.
- limit *(integer, default:* 20*)*: The maximum number of products to return per page. Limits the size of the response.
- category *(string, optional)*: Filters products by a specific category name (e.g., 'Electronics', 'Apparel').
- search *(string, optional)*: Performs a full-text search within product names and descriptions.
- min_price *(decimal, optional)*: Filters products with a price greater than or equal to this value.
- max_price *(decimal, optional)*: Filters products with a price less than or equal to this value.
- sort *(string, optional)*: Specifies the field by which to sort the results (e.g., 'name', 'price', 'created_at').
- order *(string, optional)*: Specifies the sort direction, either 'asc' (ascending) or 'desc' (descending).

Response (200 OK)

JSON


```
{
 "products": [
  {
   "id": 1,
   "name": "Wireless Headphones",
   "description": "Premium wireless audio experience",
   "image": "https://cdn.example.com/product1.jpg",
   "price": 149.99,
   "category": "Electronics",
   "created_at": "2024-01-01T00:00:00Z"
  }
 ],
 "pagination": {
  "page": 1,
  "limit": 20,
  "total": 150,
  "total_pages": 8
 }
}
```

Fields

- products *(array of objects)*: A list of product objects matching the query criteria.
    - id *(integer)*: Unique identifier of the product.
    - name *(string)*: Name of the product.
    - description *(string)*: Detailed description of the product.
    - image *(string)*: URL to the product's image.
    - price *(decimal)*: Retail price of the product.
    - category *(string)*: Category the product belongs to.
    - created_at *(string)*: ISO 8601 timestamp representing when the product was added.
- pagination *(object)*: Contains pagination metadata.
    - page *(integer)*: The current page number.
    - limit *(integer)*: The number of items per page.
    - total *(integer)*: The total number of products available matching the criteria.
    - total_pages *(integer)*: The total number of pages available.

Error Responses

400 Bad Request

Returned if any query parameter is invalid (e.g., page is not an integer, sort field does not exist).

JSON

```
{
  "error": {
    "code": "INVALID_QUERY_PARAMETER",
    "message": "Invalid value for page parameter"
  }
}
```

Technical Considerations:

- Database Indexing: Efficient database indexing on `category`, `name`, and `price` fields is crucial for the performance of filtering and sorting operations.
- Query Sanitization: All query parameters are sanitized to prevent SQL injection or other malicious attacks.
- Caching: Responses for frequently accessed product listings can be cached at the API gateway or application level to reduce database load and improve response times.
- Pagination Strategy: Offset-based pagination is used, which is suitable for moderate datasets. For very large datasets, cursor-based pagination might be considered for better performance.

POST /products (Admin Only)

This endpoint allows authorized administrators to add new products to the e-commerce catalog. It requires specific administrative privileges and a valid JWT with an 'admin' role. This ensures that only trusted personnel can modify the product inventory, maintaining data integrity and system security.

Products API Reference
Create Product
Purpose: To create a new product entry in the database. This operation is restricted to users with administrative privileges.

- **Endpoint:** /products
- **Method:** POST
- **Headers:**
  - Content-Type: application/json
  - Authorization: Bearer <JWT_Bearer_token>

Request Body

JSON

```
{
 "name": "New Product",
 "description": "Product description",
 "price": 99.99,
 "category": "Electronics",
 "image": "https://cdn.example.com/new-product.jpg",
 "inventory_count": 50
}
```

Fields

- name *(string, required)*: The name of the new product.
- description *(string, required)*: A detailed description of the product.
- price *(decimal, required)*: The price of the product. Must be a positive value.
- category *(string, required)*: The category to which the product belongs.
- image *(string, optional)*: A URL to the product image. If not provided, a default image might be assigned.
- inventory_count *(integer, required)*: The initial stock quantity of the product. Must be a non-negative integer.

Response (201 Created)

JSON

```
{
 "id": 101,
 "name": "New Product",
 "description": "Product description",
 "price": 99.99,
 "category": "Electronics",
 "image": "https://cdn.example.com/new-product.jpg",
 "inventory_count": 50,
 "created_at": "2024-01-01T12:00:00Z"
}
```

Fields

- Returns the newly created product object, including its generated id and created_at timestamp.

Error Responses

400 Bad Request

Returned if the request body is invalid (e.g., missing required fields, invalid data types, negative values).

401 Unauthorized

Returned if the Authorization header is missing or the JWT token is invalid/expired.

403 Forbidden

Returned if the authenticated user does not have administrative privileges (admin role).</JWT_Bearer_token>

Technical Considerations:

- Role-Based Access Control (RBAC): The API gateway or a dedicated middleware layer enforces that only users with the is a `admin` role can access this endpoint. This critical security measure.

- Input Validation: Comprehensive server-side validation is performed on all incoming product data to ensure data quality and prevent malicious inputs.
- Image Uploads: While the `image` field accepts a URL, a more robust solution would involve integrating with a cloud storage service (like Supabase Storage or AWS S3) for direct image uploads and management.
- Atomic Operations: Product creation, including initial inventory count, should be handled as an atomic database transaction to ensure consistency.

Order Management Endpoints

POST /orders

This endpoint allows an authenticated user to create a new order. The request body includes the list of items to be purchased, shipping address details, and the chosen payment method. Upon successful creation, the API returns the newly created order object, including a unique order ID and its initial status. This endpoint is crucial for the core e-commerce transaction flow.

Orders API Reference
Create Order

Purpose: To allow an authenticated user to place a new order, specifying the products, quantities, shipping details, and payment method.

- **Endpoint:** /orders
- **Method:** POST
- **Headers:**
  - Content-Type: application/json
  - Authorization: Bearer <JWT_Bearer_token>

Request Body

JSON

```
{
 "items": [
   {
```

```
    "product_id": 1,
    "quantity": 2,
    "price": 149.99
   },
   {
    "product_id": 3,
    "quantity": 1,
    "price": 79.99
   }
 ],
 "shipping_address": {
  "street": "123 Main St",
  "city": "Anytown",
  "state": "CA",
  "zip_code": "12345",
  "country": "USA"
 },
 "payment_method": "credit_card"
}
```

Fields

- items *(array of objects, required)*: A list of products the user wishes to purchase.
    - product_id *(integer)*: Unique identifier of the product.
    - quantity *(integer)*: Quantity of the product.
    - price *(decimal)*: Price of the product at the time of purchase.
- shipping_address *(object, required)*: Details for shipping the order.
    - Must contain street, city, state, zip_code, and country (all strings).
- payment_method *(string, required)*: The chosen payment method (e.g., 'credit_card', 'paypal').

Response (201 Created)

JSON

```
{
 "order": {
  "id": "order-uuid",
  "user_id": "user-uuid",
  "items": [
   {
    "product_id": 1,
    "quantity": 2,
    "price": 149.99
   },
   {
    "product_id": 3,
    "quantity": 1,
    "price": 79.99
   }
  ],
  "total_price": 379.97,
  "status": "confirmed",
  "created_at": "2024-01-01T12:00:00Z"
 }
}
```

Fields

- order *(object)*: The newly created order object.
    - id *(string)*: Unique identifier for the order.

- user_id *(string)*: Identifier of the user who placed the order.
- items *(array of objects)*: The list of items included in the order.
- total_price *(decimal)*: The final calculated total price of the order.
- status *(string)*: The initial status of the order (e.g., 'confirmed').
- created_at *(string)*: Timestamp of order creation.

Error Responses

400 Bad Request

Returned if the request body is invalid (e.g., missing required fields, invalid product IDs, insufficient stock, invalid shipping address).

JSON

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Product with ID 123 is out of stock"
  }
}
```

401 Unauthorized

Returned if no authentication token is provided or the token is invalid.

403 Forbidden

Returned if the authenticated user does not have permission to place orders.</JWT_Bearer_token>

Technical Considerations:

- Transactional Integrity: Order creation is a complex operation involving multiple steps (e.g., inventory deduction, payment processing, order record creation). These steps must be wrapped in a database transaction to ensure atomicity. If any step fails, the entire transaction should be rolled back.
- Inventory Management: Before creating an order, the API must verify that sufficient stock is available for all requested items. Stock levels should be decremented atomically to prevent overselling.
- Payment Processing: Integration with a payment gateway is essential. The API should initiate payment processing and only confirm the order if the payment is successful. Error handling for payment failures is critical.
- Asynchronous Processing: For complex order fulfillment processes (e.g., sending confirmation emails, updating external systems), asynchronous tasks (e.g., using message queues) can be employed to improve API response times and system resilience.
- Security: Sensitive payment information should never be stored directly on the server. Instead, tokenization or direct integration with PCI-compliant payment gateways should be used.

GET /orders

This endpoint allows authenticated users to retrieve their order history. Administrators can use this endpoint to view all orders or filter them by specific criteria. It supports pagination to efficiently handle large numbers of orders and allows filtering by order status, providing flexibility for both users and administrative staff.

Orders API Reference

Get Orders List
Purpose: To retrieve a list of orders for the authenticated user, or all orders if the requester has administrative privileges. Supports filtering and pagination.

- **Endpoint:** /orders
- **Method:** GET
- **Headers:**
    - Authorization: Bearer <JWT_Bearer_token>

Query Parameters

- status *(string, optional)*: Filters orders by their current status (e.g., 'open', 'ongoing', 'delivered').
- page *(integer, default:* 1*)*: The page number of the results to retrieve. Used for pagination.
- limit *(integer, default:* 10*)*: The maximum number of orders to return per page. Limits the size of the response

Response (200 OK)

JSON


```
{
 "orders": [
  {
   "id": "order-uuid",
   "user_id": "user-uuid",
   "items": [
    {
     "product_id": 1,
     "quantity": 2,
     "price": 149.99
    }
   ],
   "total_price": 379.97,
   "status": "confirmed",
   "created_at": "2024-01-01T12:00:00Z",
   "updated_at": "2024-01-01T12:30:00Z"
  }
 ],
 "pagination": {
  "page": 1,
  "total": 25,
  "total_pages": 3
 }
}
```


Fields

- orders *(array of objects)*: A list of order objects matching the query criteria.
    - id *(string)*: Unique identifier for the order.
    - user_id *(string)*: Identifier of the user who placed the order.
    - items *(array of objects)*: The list of items included in the order.
    - total_price *(decimal)*: The final calculated total price of the order.
    - status *(string)*: The current status of the order.
    - created_at *(string)*: Timestamp of order creation.
    - updated_at *(string)*: Timestamp of the last order update.
- pagination *(object)*: Contains pagination metadata.
    - page *(integer)*: The current page number.
    - total *(integer)*: The total number of orders available matching the criteria.
    - total_pages *(integer)*: The total number of pages available.

Error Responses

400 Bad Request

Returned if any query parameter is invalid (e.g., status is not a valid order status).

JSON

```
{
  "error": {
    "code": "INVALID_QUERY_PARAMETER",
    "message": "Invalid value for status parameter"
  }
}
```

401 Unauthorized

Returned if no authentication token is provided or the token is invalid.

403 Forbidden

Returned if a non-admin user attempts to access orders that do not belong to them.</JWT_Bearer_token>

Technical Considerations

Row Level Security (RLS)

For non-admin users, RLS policies on the table ensure that they can only retrieve orders associated with their user_id, preventing unauthorized access to other users' data.

Performance Optimization

Efficient indexing on user_id and status fields is crucial for fast retrieval of order lists, especially for users with many orders or when filtering by status.

Data Projection

The API should carefully select which fields to return in the order objects to avoid exposing sensitive information unnecessarily

Audit Trails

The created_at and updated_at timestamps provide valuable information for auditing and tracking changes to order records.

API Security and Standards

Authentication

Authentication is a cornerstone of API security, ensuring that only legitimate and authorized users can access protected resources. The e-commerce API employs a robust JWT (JSON Web Token) Bearer Token authentication scheme, providing a stateless and scalable method for verifying user identities.

- **JWT Bearer Tokens** in Authorization header: All authenticated requests to protected API endpoints must include a JWT in the `Authorization` header, prefixed with `Bearer`. This token is issued upon successful login and contains claims about the user's identity and permissions.
- **Token Expiration** with automatic refresh: Access tokens are designed to be short-lived to minimize the window of opportunity for attackers if a token is compromised. A longer-lived refresh token is provided to the client, allowing them to obtain new access tokens without requiring the user to re-authenticate frequently. This enhances both security and user experience.
- **Role-based Access** control for admin endpoints: The JWT includes a `role` claim (e.g., 'user', 'admin'). The API gateway or a dedicated middleware checks this claim to enforce role-based access control (RBAC), ensuring that only users with the appropriate roles (e.g., 'admin') can access sensitive administrative endpoints.

Rate Limiting

Rate limiting is a crucial security measure implemented to protect the API from abuse, such as brute-force attacks, denial-of-service (DoS) attempts, and excessive usage that could degrade performance for legitimate users. The API employs a token bucket or leaky bucket algorithm to enforce these limits, ensuring fair resource allocation.

Endpoint Type	Limit	Window	Description
Authentication	5 requests	15 minutes	Strict limits on login/registration to prevent brute-force password attacks.

Product Queries	100 requests	15 minutes	Higher limits for browsing products, allowing for normal user exploration.
Order Operations	20 requests	15 minutes	Moderate limits to prevent automated order creation or manipulation.
Admin Operations	50 requests	15 minutes	Sufficient limits for administrative tasks while preventing abuse.

When a client exceeds the allowed rate limit, the API responds with a 429 Too Many Requests status code, indicating that the client should back off and retry later. The response may also include a Retry-After header specifying the time to wait before making another request.

Error Response Format

Consistent and informative error responses are crucial for API usability and debugging. The e-commerce API adheres to a standardized error response format, providing developers with clear, actionable information when an error occurs. This format includes a unique error code, a human-readable message, and optional detailed information to pinpoint the exact cause of the issue.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input
    parameters", "details": {
      "field": "email",
      "reason": "Invalid email
}
```

- `error` (object): Contains details about the error.
  - `code` (string): A unique, application-specific error code (e.g., `VALIDATION_ERROR`, `UNAUTHORIZED`, `PRODUCT_NOT_FOUND`). This allows programmatic handling of different error types.
  - `message` (string): A human-readable description of the error, providing context to the developer.
  - `details` (object, optional): An optional object containing more specific information about the error, such as the field that failed validation or specific

reasons for the failure.

- `timestamp` (string): The UTC timestamp when the error occurred, useful for logging and debugging.
- `request_id` (string): A unique identifier for the request that caused the error, facilitating tracing and support.

Best Practices for Error Handling:

- Predictable Structure: Always return errors in a consistent JSON structure, making it easier for client applications to parse and handle them.
- Meaningful Codes: Use specific error codes that clearly indicate the type of error, rather than generic messages.
- Developer-Friendly Messages: Error messages should be clear, concise, and provide enough information for developers to understand and resolve the issue.
- Logging: All errors are logged on the server-side with their quick diagnosis and resolution of issues.

HTTP Status Codes

Code	Meaning	Usage
200	OK	Successful GET requests
201	Created	Successful resource creation
400	Bad Request	Validation errors
401	Unauthorized	Missing/invalid authentication
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server-side errors
